

ECS Project Step 4 – Authentication with EC2 and Pushing Image to ECR

To Begin with the Lab

Summary of the Lab

In this combined lab step, the build machine was prepared to work with **Amazon ECR** by creating and attaching an **IAM** role with **AmazonEC2ContainerRegistryFullAccess** to the **Amazon EC2** instance. The Docker client on the build machine was then authenticated to the private ECR repository using the push command shown in **Amazon ECR**, and the login completed successfully. After authentication, the local Docker image `cloudfreeks-s3-php` was verified, tagged with the full ECR repository URI, and pushed to the private repository. Finally, the repository was refreshed to confirm that the image was now present in **Amazon ECR** and ready for **Amazon ECS** to use later.

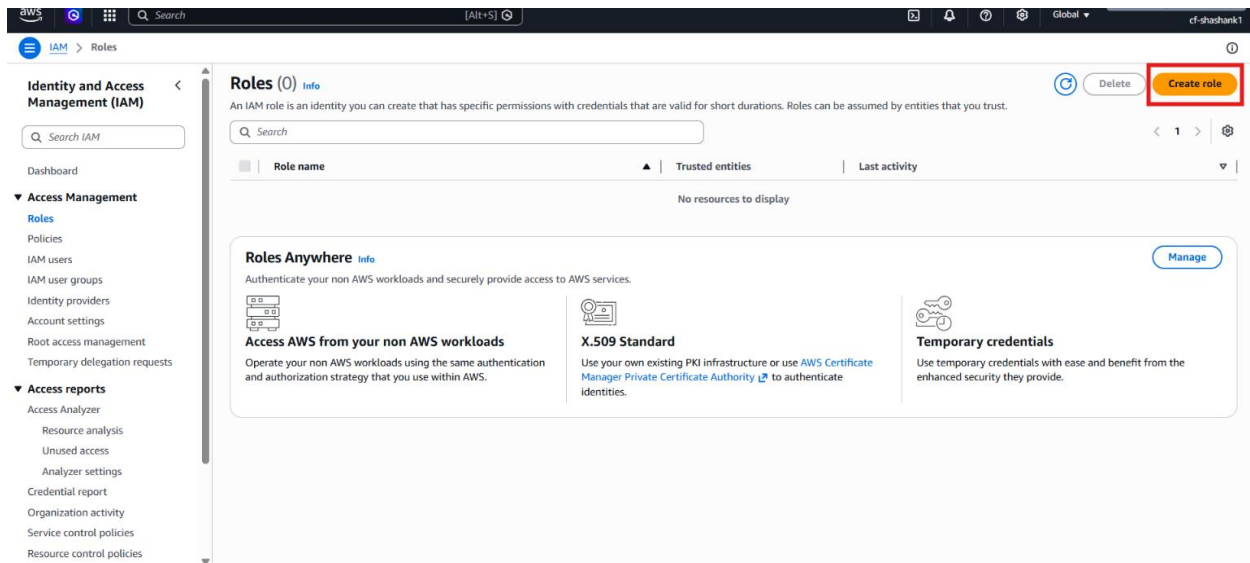
Understanding Amazon ECR Authentication, Tagging, and Pushing

Amazon ECR is a private container registry, so the build machine must first be allowed to talk to it before any image can be uploaded. The **IAM** role gives the EC2 build machine temporary permission to use ECR commands without storing AWS credentials inside the server, and Docker login proves that the client is authenticated. Tagging is needed because the local image name is not enough for ECR; Docker must know the exact repository URI where the image should be sent. For example, `cloudfreeks-s3-php:latest` is retagged with the ECR URI so the same image can be pushed into the private registry and later pulled by **Amazon ECS** during deployment.

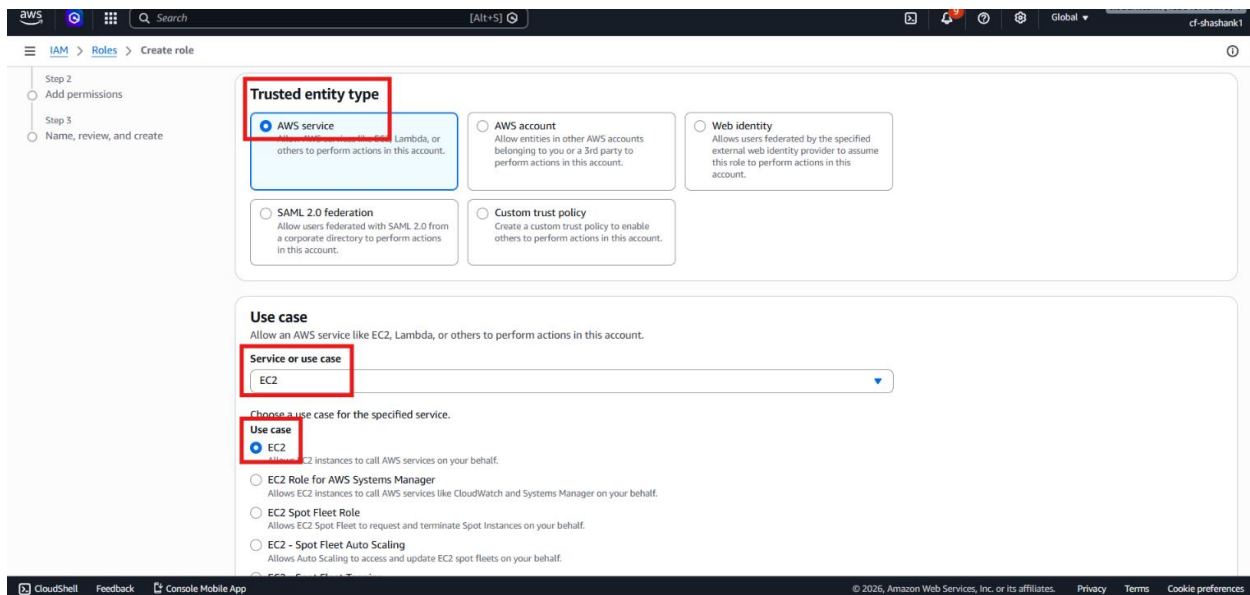
Lab Steps

Step 1 – Create the IAM Role for ECR Access

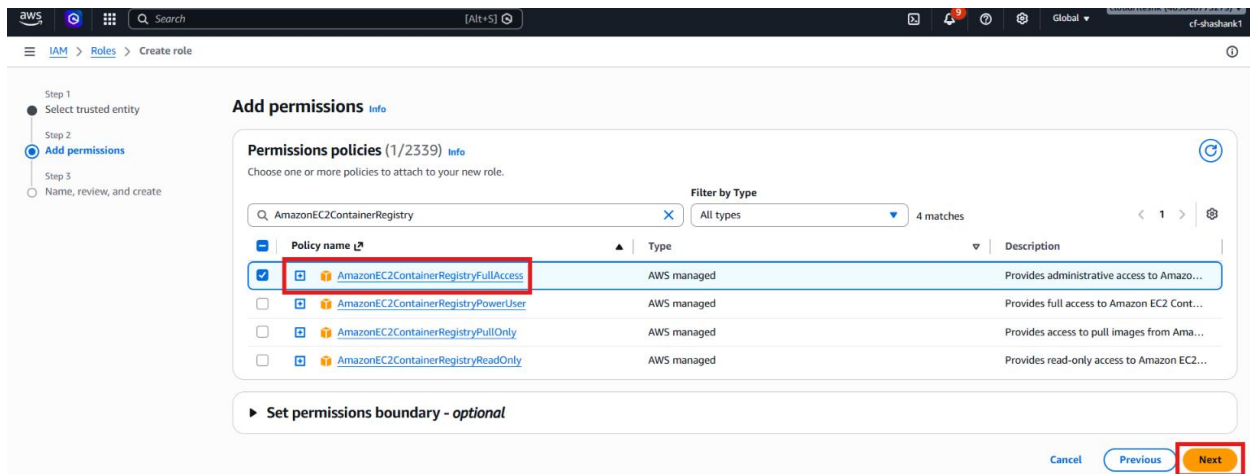
- Sign in to the **AWS Management Console**.
- Navigate to **IAM** → **Roles**.
- Click **Create role**.



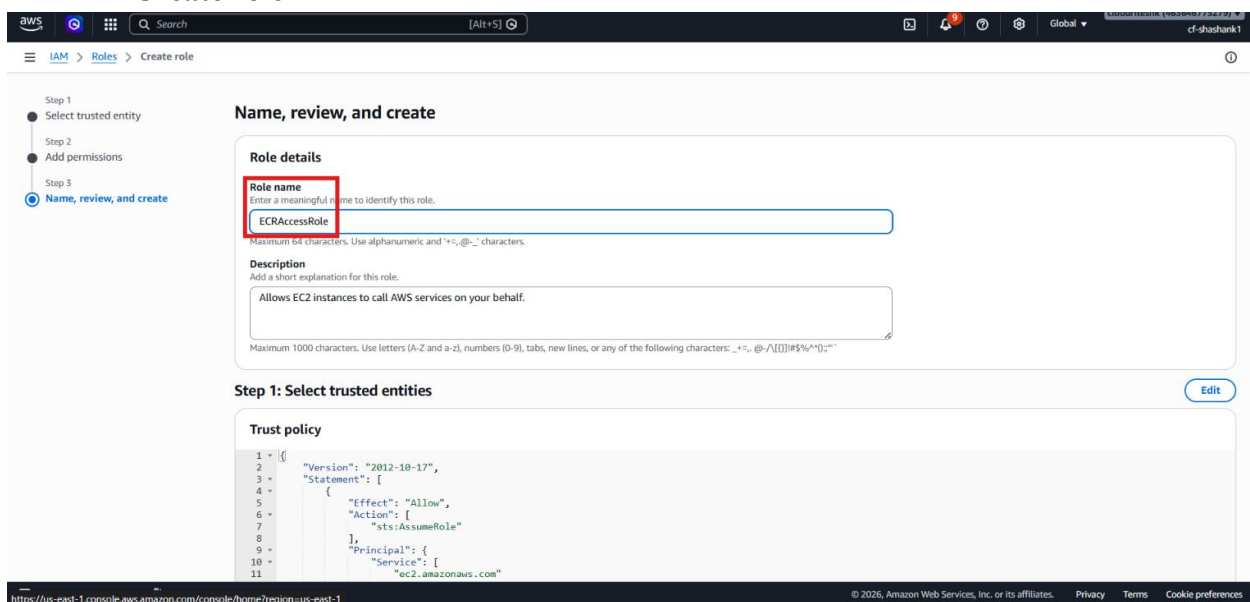
- Choose **AWS service** as the trusted entity type.
- Select **EC2** as the use case.



- Attach the **AmazonEC2ContainerRegistryFullAccess** policy.

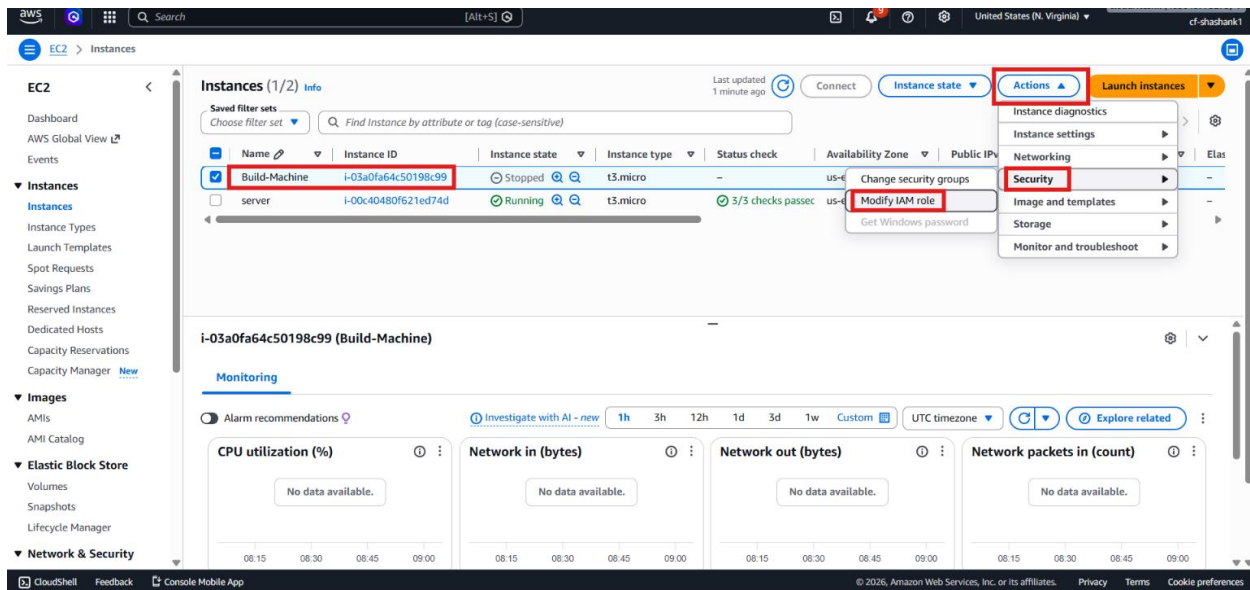


- Name the role ECRAccessRole.
- Click **Create role**.

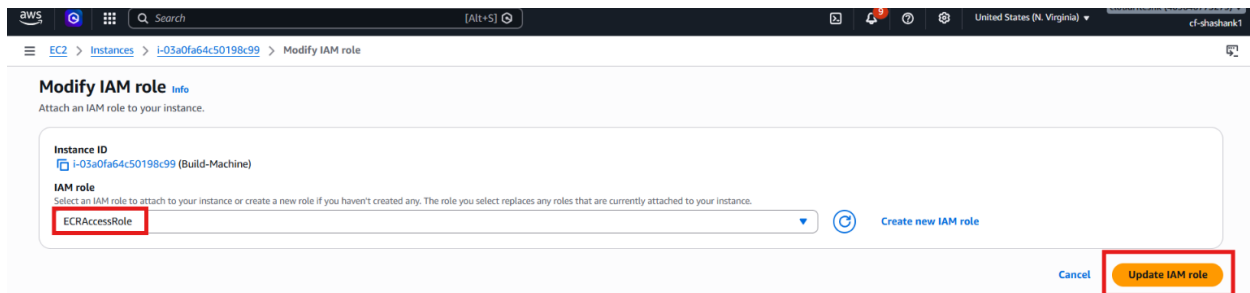


Step 2 – Attach the Role to the Build Machine

- Open **Amazon EC2**.
- Select the Docker build machine instance.
- Click **Actions** → **Security** → **Modify IAM role**.

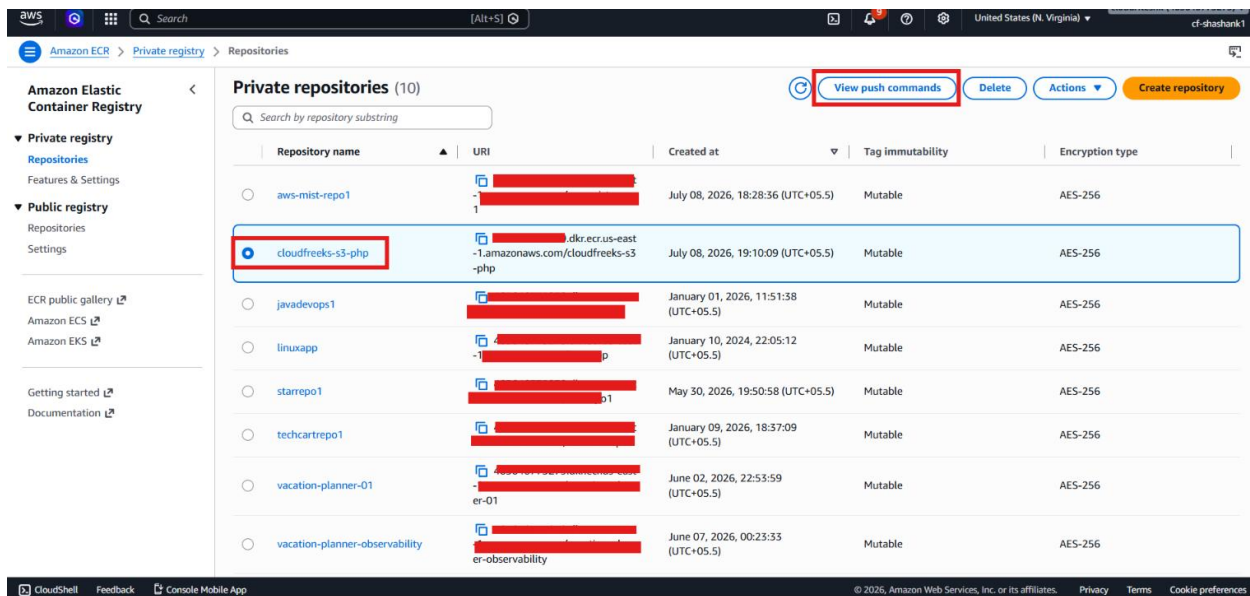


- Attach the ECRAccessRole to the instance.

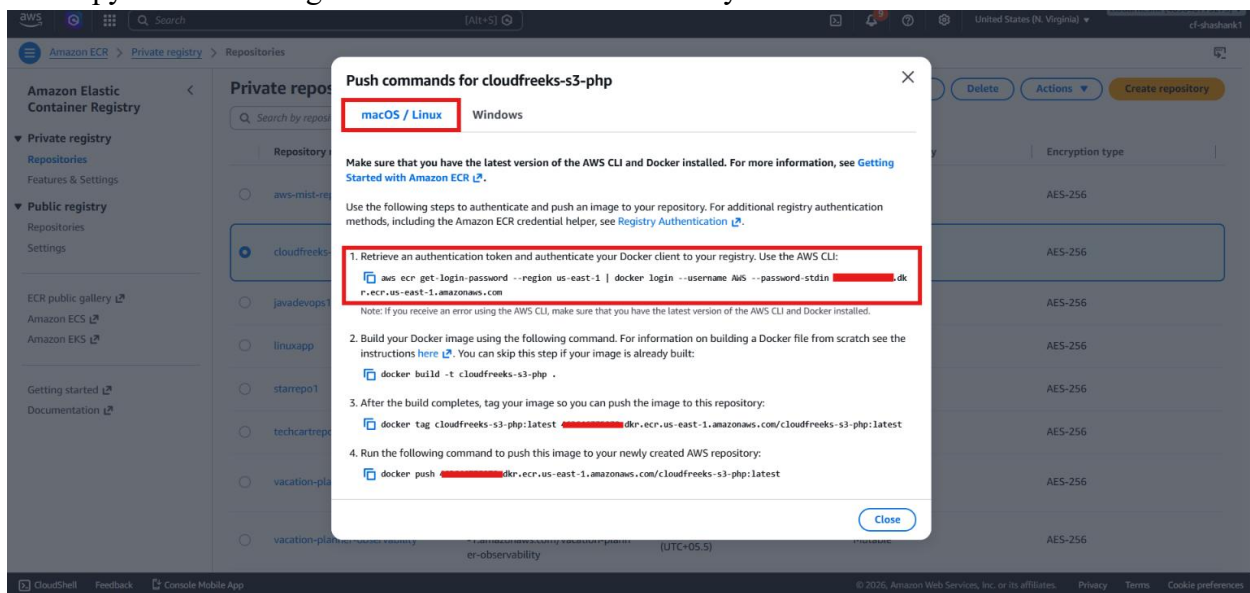


Step 3 – Authenticate Docker with Amazon ECR

- Start and Connect to the EC2 Instance for this step if you have not started it yet.
- Open **Amazon ECR**.
- Open the repository **cloudfreeks-s3-php**.
- Click **View push commands**.



- Copy the Docker login command shown for Linux systems.

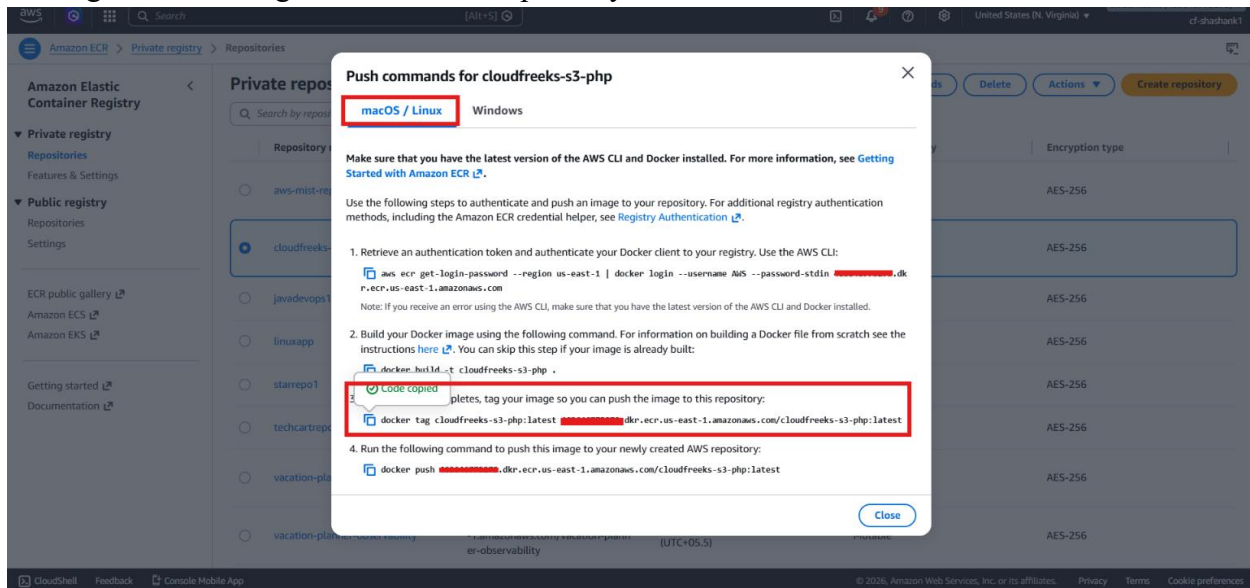


- Run the command on the build machine.
- Confirm that the login completes successfully.

- Verify that cloudfreeks-s3-php appears in the list.

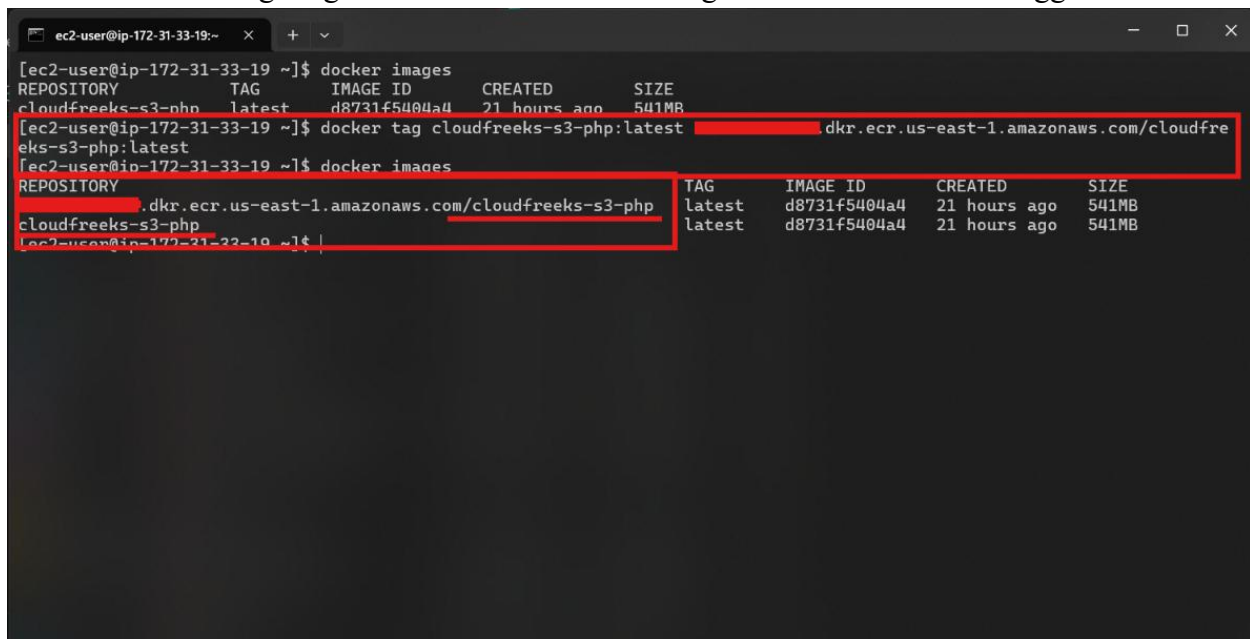
Step 6 – Tag the Docker Image

- Copy the repository URI from **Amazon ECR**.
- Tag the local image with the ECR repository URI.



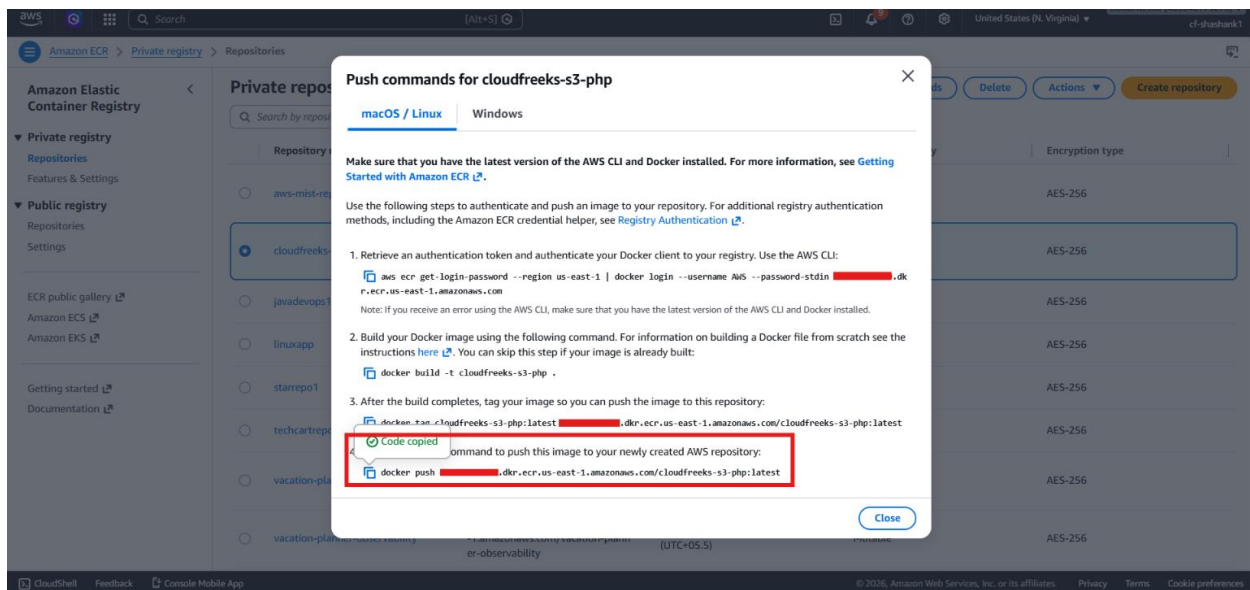
```
docker tag cloudfreeks-s3-php:latest <account-id>.dkr.ecr.<region>.amazonaws.com/cloudfreeks-s3-php:latest
```

- Run docker images again and confirm that the image now shows the ECR-tagged name.



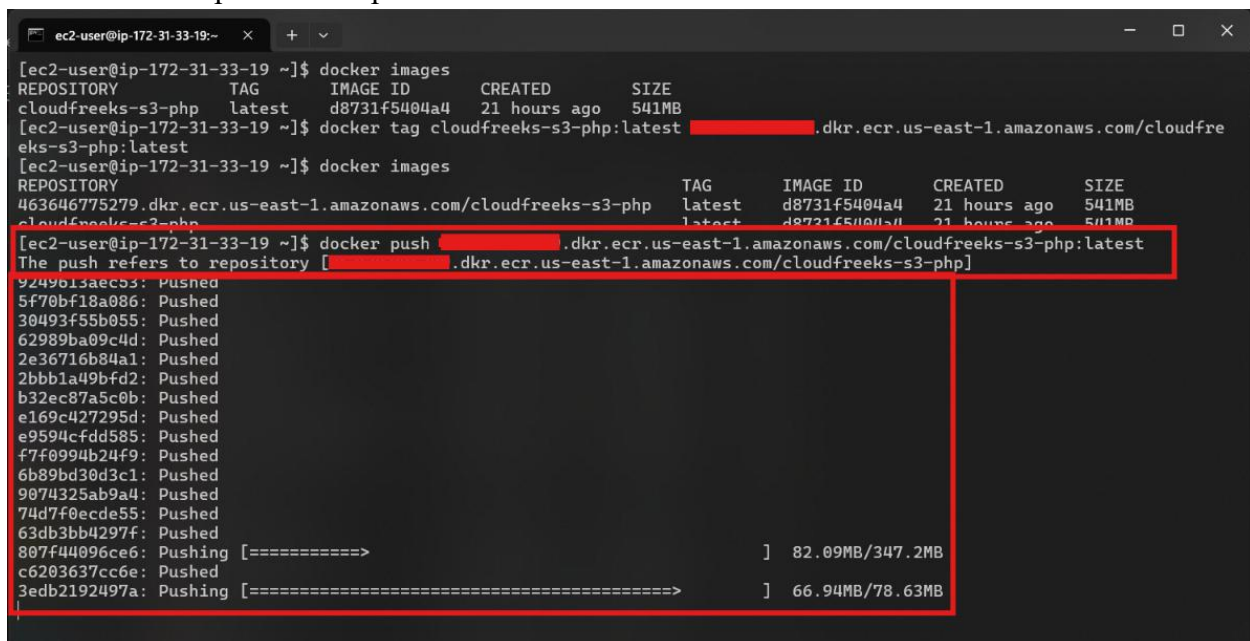
Step 7 – Push the Image to Amazon ECR

- Run the Docker push command shown in **View push commands**.



`docker push <account-id>.dkr.ecr.<region>.amazonaws.com/cloudfreeks-s3-php:latest`

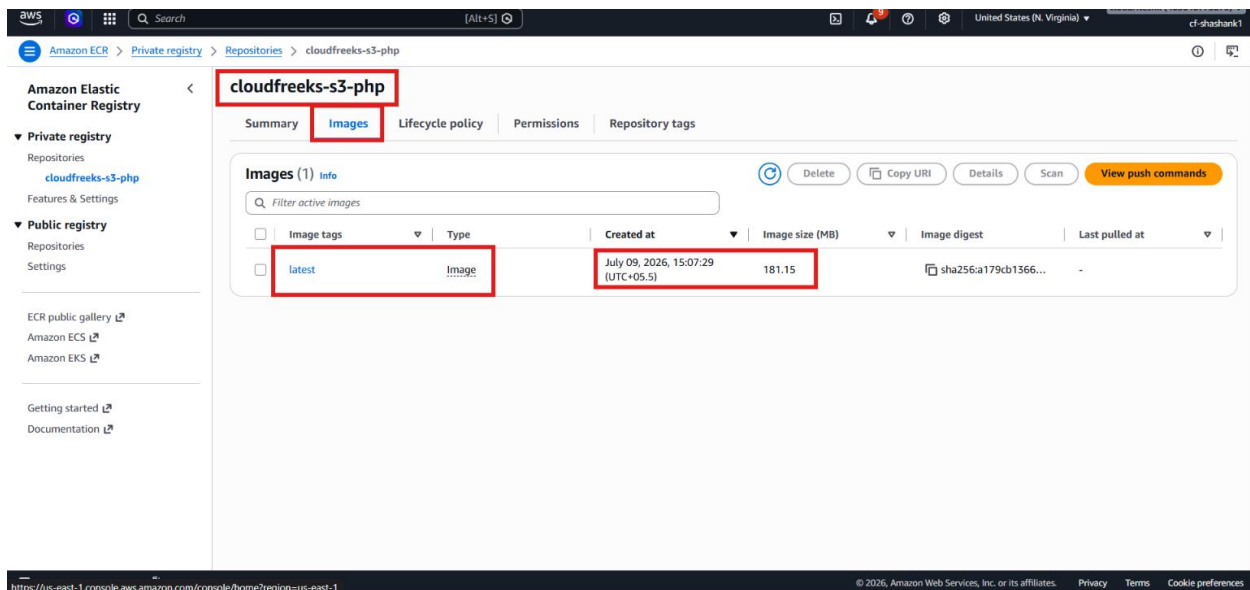
- Wait for the push to complete.



- Confirm that Docker reports the upload successfully.

Step 8 – Verify the Image in ECR

- Return to the **Amazon ECR** repository.
- Refresh the page.
- Confirm that the image is now listed in the repository.



- Verify that the tag, digest, size, and push time are visible.

Verification

- Verified that the **ECRAccessRole** was created and attached to the build machine.
- Verified that the Docker login to **Amazon ECR** completed successfully.
- Verified that the local image cloudfreeks-s3-php existed on the build machine.
- Verified that the image was tagged with the correct ECR repository URI.
- Verified that the docker push command uploaded the image successfully.
- Verified that the image appeared in the **Amazon ECR** repository after refresh.